2. HotStuff 핵심 데이터 구조

2.1 기본 타입 정의

2.1.1 식별자와 뷰

```
// 복제본 고유 식별자
type ID uint32

// 뷰 번호 (합의 라운드)
type View uint64

// SHA256 해시
type Hash [32]byte

// 클라이언트 명령
type Command string
```

2.1.2 복제본 집합 (IDSet)

```
type IDSet interface {
Add(id ID) // ID 추가
```

```
Contains(id ID) bool  // ID 포함 여부 확인

ForEach(f  func(ID)) // 모든 ID에 대해 함수 실행

RangeWhile(f  func(ID) bool) // 조건부 순회

Len() int  // 집합 크기

}
```

구현 특징:

- 맵 기반 구현으로 O(1) 검색 성능
- 쿼럼 계산과 서명 참여자 추적에 사용
- 스레드 안전성은 상위 레벨에서 보장

2.2 블록 구조 (Block)

2.2.1 블록 정의

```
type Block struct {

hash Hash // 블록 해시 (캐시됨)

parent Hash // 부모 블록 해시

proposer ID // 제안자 ID

cmd Command // 실행할 명령

cert QuorumCert // 쿼럼 인증서

view View // 뷰 번호

ts time.Time // 타임스탬프

}
```

2.2.2 블록 생성과 해시

```
func  NewBlock(parent Hash, cert QuorumCert, cmd Command, view View, proposer ID

b := &Block{

parent: parent,

cert: cert,

cmd: cmd,

view: view,

proposer: proposer,

ts: time.Now(),

}

// 해시를 즉시 계산하여 캐시 (동시성 문제 방지)

b.hash = sha256.Sum256(b.ToBytes())

return  b

}
```

2.2.3 블록 직렬화

```
func (b *Block) ToBytes() []byte {

buf := b.parent[:] // 부모 해시

buf = append(buf, b.proposer.ToBytes()...) // 제안자 ID

buf = append(buf, b.view.ToBytes()...) // 뷰 번호

buf = append(buf, []byte(b.cmd)...) // 명령
```

```
buf = append(buf, b.cert.ToBytes()...) // QC

buf = append(buf, timestampToBytes(b.ts)...) // 타임스탬프

return  buf

}
```

2.3 인증서 시스템

2.3.1 부분 인증서 (PartialCert)

```
type PartialCert struct {

signer ID // 서명자 ID (최적화용)

signature QuorumSignature // 서명

blockHash Hash // 서명된 블록 해시

}
```

특징:

- 개별 복제본의 블록 승인 표시
- 서명자 ID를 별도 저장하여 빠른 접근
- 쿼럼 인증서 생성의 기본 단위

2.3.2 쿼럼 인증서 (QuorumCert)

```
type QuorumCert struct {

signature QuorumSignature // 집계된 서명
```

```
view View // 인증서 생성 뷰
hash Hash // 인증된 블록 해시
}
```

역할:

- 쿼럼(2f+1)의 복제본이 블록을 승인했음을 증명
- 다음 블록의 부모 증명으로 사용
- 체인 안전성의 핵심 메커니즘

2.3.3 타임아웃 인증서 (TimeoutCert)

```
type TimeoutCert struct {
signature QuorumSignature // 타임아웃 서명들
view View // 타임아웃된 뷰
}
```

용도:

- 뷰에서 진행이 멈췄을 때 다음 뷰로 이동
- 활성(liveness) 보장의 핵심 메커니즘
- 쿼럼의 타임아웃 메시지로 생성

2.4 동기화 정보 (SyncInfo)

2.4.1 SyncInfo 구조

```go
type SyncInfo struct {
qc *QuorumCert // 최고 QC
tc *TimeoutCert // 최고 TC
aggQC *AggregateQC // 집계 QC (Fast-HotStuff용)
}
```

2.4.2 빌더 패턴 구현

```go
func  NewSyncInfo() SyncInfo {
return SyncInfo{}
}

func (si SyncInfo) WithQC(qc QuorumCert) SyncInfo {
si.qc = &qc
return  si
}

func (si SyncInfo) WithTC(tc TimeoutCert) SyncInfo {
si.tc = &tc
return  si
}
```

설계 원칙:

- 불변성(immutability) 보장
- 메서드 체이닝으로 편리한 구성
- 옵셔널 필드들의 안전한 처리

2.5 Fast-HotStuff 전용 구조

2.5.1 집계 QC (AggregateQC)

```
type AggregateQC struct {
qcs  map[ID]QuorumCert // 각 복제본의 QC
sig QuorumSignature // 집계된 타임아웃 서명
view View // 생성 뷰
}
```

특징:

- 타임아웃 메시지에서 QC들을 추출하여 집계
- 2단계 체인으로 커밋 시간 단축
- 포킹 공격 방지 메커니즘 제공

2.5.2 AggregateQC 검증 과정

```
func (c *crypto) VerifyAggregateQC(aggQC AggregateQC) (highQC QuorumCert, ok  bo
// 1. 각 QC에서 최고 QC 찾기
for  _, qc := range  aggQC.QCs() {
if  highQC.View() < qc.View() {
```

```
highQC = qc

}

}

// 2. 타임아웃 메시지 재구성

messages := make(map[ID][]byte)

for  id, qc := range  aggQC.QCs() {

messages[id] = reconstructTimeoutMsg(id, aggQC.View(), qc)

}

// 3. 배치 서명 검증

if !c.BatchVerify(aggQC.Sig(), messages) {

return QuorumCert{}, false

}

// 4. 최고 QC 검증

return  highQC, c.VerifyQuorumCert(highQC)

}
```

2.6 메시지 타입들

2.6.1 제안 메시지 (ProposeMsg)

```
type ProposeMsg struct {

ID ID // 제안자 ID

Block *Block // 제안된 블록
```

```
AggregateQC *AggregateQC // 집계 QC (옵셔널)
}
```

2.6.2 투표 메시지 (VoteMsg)

```
type VoteMsg struct {
ID ID // 투표자 ID
PartialCert PartialCert // 부분 인증서
}
```

2.6.3 타임아웃 메시지 (TimeoutMsg)

```
type TimeoutMsg struct {
ID ID // 송신자 ID
View View // 타임아웃된 뷰
SyncInfo SyncInfo // 동기화 정보
ViewSignature QuorumSignature // 뷰 서명
MsgSignature QuorumSignature // 메시지 서명 (AggQC용)
}
```

2.6.4 새 뷰 메시지 (NewViewMsg)

```
type NewViewMsg struct {
ID ID // 송신자 ID
```

```
SyncInfo SyncInfo // 동기화 정보

}
```

2.7 데이터 구조 최적화

2.7.1 해시 캐싱

- 블록 해시를 생성 시점에 계산하여 저장
- 반복적인 해시 계산 비용 제거
- 동시성 환경에서 안전성 보장

2.7.2 서명자 ID 캐싱

- PartialCert에서 서명자 ID를 별도 저장
- 서명 참여자 집합 순회 없이 빠른 접근
- 메모리 사용량 약간 증가하지만 성능 향상

2.7.3 불변성 설계

- 생성 후 수정 불가능한 구조
- 동시성 환경에서 안전성 보장
- 복사 비용 최소화

2.8 직렬화와 네트워크 전송

2.8.1 바이너리 직렬화

```go
func (v View) ToBytes() []byte {
var viewBytes [8]byte
binary.LittleEndian.PutUint64(viewBytes[:], uint64(v))
return viewBytes[:]
}

func (id ID) ToBytes() []byte {
var idBytes [4]byte
binary.LittleEndian.PutUint32(idBytes[:], uint32(id))
return idBytes[:]
}
```

2.8.2 프로토콜 버퍼 변환

- gRPC 통신을 위한 protobuf 메시지 변환
- 네트워크 효율성과 언어 간 호환성 제공
- 자동 코드 생성으로 유지보수성 향상

이러한 데이터 구조들은 HotStuff 알고리즘의 안전성, 활성, 그리고 성능을 보장하는 핵심 요소들입니다. 각 구조는 특정 역할을 수행하며, 전체 시스템의 정확성과 효율성에 기여합니다.